

Optimización de Caminos en Grafos de Gran Escala: Una Aplicación de Dijkstra y BFS en la Industria Cinematográfica

Resumen

El presente trabajo explora la aplicación práctica de la Teoría de Grafos y el Algoritmo de Dijkstra para resolver problemas de conectividad en redes sociales masivas, específicamente en la base de datos de IMDb. El proyecto, titulado "Bacon Director's Cut", no solo busca el camino más corto entre actores, sino que introduce funciones de costo variables (calidad cinematográfica y popularidad) para alterar la topología del grafo dinámicamente y crear un sistema de recomendación. Se detalla la ingeniería de datos necesaria para procesar *Big Data* en entornos de memoria limitada, la arquitectura de software web basada en Flask para la interacción en tiempo real y las estrategias de optimización para el despliegue en la nube, además de la implementación de algoritmos como BFS Bidireccional y Dijkstra.

1. Introducción y Definición del Problema

¿Hay un hilo invisible que conecta toda la industria cinematográfica? La teoría de los "Seis Grados de Separación" postula que cualquier persona en la Tierra puede estar conectada a cualquier otra a través de una cadena de no más de cinco intermediarios. En el contexto cinematográfico, esto se conoce como los "Grados de Bacon". Si bien los enfoques tradicionales suelen tratar todas las conexiones por igual (grafos no ponderados), el objetivo de esta investigación es trascender la búsqueda de caminos por mera proximidad (BFS) e implementar un sistema que valore la "calidad" del camino, permitiendo al usuario priorizar rutas a través de películas de alto *rating* o películas menos conocidas. Para hacer esto, se aplicará el Algoritmo de Dijkstra sobre un grafo ponderado construido a partir de la base de datos de IMDb. Sin embargo, hay que tener en cuenta sus limitaciones, ya que IMDb sólo ofrece información sobre el reparto principal de las películas y no de todos el reparto secundario y extras. Además, los datos fueron descargados el 16 de noviembre de 2025 y no fueron actualizados desde entonces.

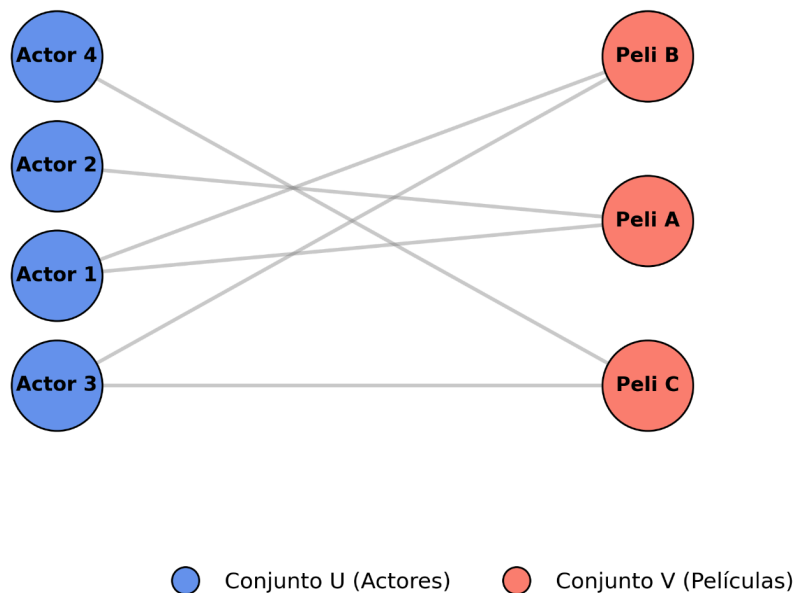
2. Abstracción y Modelado Matemático

2.1. Definición Formal del Grafo

El sistema modela la industria cinematográfica como un **Grafo Bipartito No Dirigido y Ponderado**, denotado como $G = (U, V, E, w)$. En esta estructura el conjunto total de vértices ($U \cup V$) se particiona en dos subconjuntos disjuntos ($U \cap V = \emptyset$), donde 'U' representa a los actores (identificados por $nconst$) y 'V' representa a las películas (identificadas por $tconst$). La estructura de la topología queda definida por el conjunto de aristas (E), donde existe una relación $e = \{u, v\} \Leftrightarrow$ el actor $u \in U$ participó en la película $v \in V$. Esta configuración preserva la estructura bipartita, lo que implica que la relación no es reflexiva ni transitiva, dado que no existen adyacencia directas entre vértices de la misma clase, aunque sí es simétrica porque la conexión es no dirigida.

Para la manipulación algorítmica eficiente, el grafo se implementa lógicamente mediante *Listas de Adyacencia* utilizando tablas hash (diccionarios de Python). Esto se traduce en aristas bidireccionales que permiten el recorrido en cualquier sentido.

Modelo Bipartito $G = (U, V, E)$



2.2. Funciones de Peso

A diferencia de un grafo estático, este proyecto implementa una función de peso ($w: E \Rightarrow \mathbb{R}^+$) dinámica, que varía según el perfil de búsqueda seleccionado por el usuario.

Esto permite que el mismo conjunto de datos (películas y actores) se comporte como topologías distintas dependiendo de la intención de la búsqueda.

1. Modo Casual (Ponderación Lineal):

Este perfil modela a un espectador promedio que busca evitar películas de mala calidad, pero que no penaliza la popularidad. La función de costo se define como una relación lineal inversa al *rating*:

$$w_{casual}(e) = 10.1 - Rating(v)$$

2. Modo Crítico (Ponderación Exponencial y Logarítmica):

Este perfil modela el comportamiento de un cinéfilo exigente que valora las "joyas ocultas" y rechaza activamente el cine comercial masivo, priorizando la excelencia sobre la proximidad y equilibrando el grafo hacia películas de alto culto. Para lograr este comportamiento no lineal, se diseñó una función de costo compuesta:

$$w_{crítico}(e) = FactorCalidad \times FactorFama$$

$$w_{crítico}(e) = 2,5^{10 - Rating(v)} \times (\log_{10}(Votos))^2$$

Esta formulación matemática integra dos mecanismos de penalización no lineal que redefinen la ponderación del grafo. En primer lugar, el 'Factor de Calidad' abandona la progresión lineal del modo casual en favor de una función exponencial de base 2.5, donde el exponente está determinado por la distancia a la puntuación perfecta; esto implica que el costo de transitar por una arista aumenta drásticamente ante la más mínima caída en el *rating*. Simultáneamente, se introduce un 'Factor de Fama' modelado mediante el cuadrado del logaritmo de la cantidad de votos, variable utilizada como indicador de la popularidad masiva. Este componente encarece artificialmente las aristas de las producciones *mainstream*, forzando al sistema a descartar los caminos populares y explorar rutas topológicas alternativas de menor exposición.

2.3. Filtrado Topológico Dinámico (Máscaras de Inclusión)

En un inicio En las implementaciones tradicionales de grafos, cuando se aplican filtros restrictivos (por ejemplo, "Solo películas de Acción de los 90"), suele utilizarse una estrategia de reconstrucción: se crea un subgrafo nuevo en memoria que contiene únicamente los nodos que cumplen con los criterios. Si bien esto garantiza la consistencia, el costo computacional de construir una nueva estructura para cada consulta es prohibitivo

en sistemas de gran escala. Para resolver esto, el proyecto implementa una estrategia de Enmascaramiento Lógico (Topological Masking). El grafo estructural permanece inmutable y completo en la base de datos. El filtrado ocurre mediante la definición dinámica de un conjunto de elementos válidos.

El mecanismo de filtrado se operacionaliza mediante un 'Proceso de Máscara' que evita la reconstrucción costosa del grafo. La estrategia comienza con la definición rigurosa de un conjunto de validación 'S', generado a partir de una consulta optimizada que recupera exclusivamente los identificadores de películas que satisfacen los criterios de año, género, *rating* y votos.

$$S = \text{IDs de películas que satisfacen Año, Género, Rating y Votos}$$

Posteriormente, este conjunto S se inyecta en los algoritmos de búsqueda (BFS o Dijkstra) funcionando como una lista de admisión dinámica. Durante la navegación topológica, cada intento de cruzar una arista se somete a una verificación de pertenencia en tiempo de ejecución: si el nodo destino es un elemento de S ($v \in S$), se considera activo y permite el paso; en caso contrario, el nodo se trata como bloqueado, forzando al algoritmo a descartar esa ruta y calcular caminos alternativos.

Esta técnica transforma un problema de Construcción de Grafos (que requiere iterar y copiar millones de datos) en un problema de Verificación de Pertenencia en un Conjunto Hash (Set). En términos de eficiencia, verificar si un ID existe en un conjunto es una operación instantánea, lo que permite aplicar filtros complejos en tiempo real sin degradar la velocidad de respuesta del sistema.

2.4. Implementación de los Algoritmos de Búsqueda

El sistema adopta un enfoque híbrido, seleccionando el algoritmo más eficiente según la naturaleza de la consulta (topológica vs. ponderada).

2.4.1. Búsqueda Bidireccional (Modo Velocidad)

Para las consultas donde el objetivo es minimizar la cantidad de intermediarios (aristas no ponderadas), se implementó una Búsqueda en Anchura (BFS) Bidireccional. El proceso inicia definiendo dos fronteras de exploración simultáneas: una desde el nodo origen S y otra desde el nodo destino T . En lugar de explorar el grafo como un círculo

expansivo gigante desde el origen (lo que generaría un crecimiento exponencial de nodos visitados), se expanden dos círculos más pequeños hasta que se tocan.

La ejecución lógica del algoritmo se estructura en tres fases continuas. En la etapa de inicialización, se definen dos conjuntos de fronteras (*frontera_inicio* y *frontera_fin*) junto con sus respectivos diccionarios de procedencia, necesarios para la reconstrucción posterior del camino.

El núcleo del proceso es un bucle de expansión alternada que se mantiene activo mientras ambas fronteras contengan elementos. En cada iteración, para optimizar el rendimiento, el algoritmo evalúa ambos conjuntos y selecciona aquel con menor cantidad de nodos para expandir; esta heurística de balanceo resulta crítica para evitar explorar la filmografía de actores con una extensa carrera. Finalmente, la condición de parada se activa mediante una verificación de intersección: si un nodo recién descubierto en la frontera activa ya se encuentra registrado en la frontera opuesta, se confirma el hallazgo del camino óptimo y el algoritmo detiene su ejecución de inmediato.

2.4.2. Algoritmo de Dijkstra (Modos Ponderados)

Para los perfiles "Casual" y "Crítico", donde cada arista tiene un costo variable (función de calidad y popularidad), se utiliza el Algoritmo de Dijkstra optimizado con colas de prioridad. Sin embargo, para optimizar la búsqueda, no implementé el clásico Dijkstra sino un Dijkstra bidireccional.

El proceso inicia estableciendo la distancia a los dos orígenes (el nodo de Inicio y el nodo de Fin), que es cero para ambos. Para optimizar el espacio de memoria y evitar iterar sobre nodos irrelevantes, las distancias a todos los demás nodos se asumen predeterminadamente como infinitas hasta que son explorados.

Se utilizan estructuras de datos duplicadas para las dos búsquedas: dos Colas de Prioridad (Min-Heaps), PQ_{Fwd} y PQ_{Rev} , para almacenar los nodos por explorar, ordenados por su costo acumulado más bajo desde su respectivo origen.

El bucle principal (*while pq_fwd and pq_rev*) asegura que el algoritmo continúe mientras existan caminos candidatos por explorar en ambas direcciones. En cada iteración, el algoritmo emplea una estrategia de balanceo, extrayendo el nodo con la distancia más baja de la cola, correspondiente al "radio" de exploración más corto. Este nodo extraído se

convierte en el *nodo_actual* y se marca inmediatamente como visitado en su respectivo conjunto.

Para cada vecino, se calcula la distancia tentativa:

$$distancia_tentativa = distancia_actual + peso$$

Si esta distancia tentativa es menor que la distancia ya registrada para ese vecino en esa dirección, se actualiza el valor y el nuevo par (distancia,vecino) se inserta en el *heap* mediante *heappush()*. Este proceso garantiza que, al extraer un nodo, la distancia que posee sea la mínima final desde su punto de inicio.

La detección de encuentro es crucial: si el *nodo_actual* que se está expandiendo ya ha sido visitado o tiene una distancia registrada por la búsqueda opuesta, significa que los dos caminos se han cruzado. Se calcula el costo total del camino candidato (suma de las distancias desde Inicio y Fin) y se actualiza la variable global μ si se ha encontrado un camino mejor.

Finalmente, el algoritmo se detiene cuando se cumple la condición matemática crítica (el criterio de parada):

$$Top(PQ_{Fwd}) + Top(PQ_{Rev}) \geq \mu$$

Al ser "codicioso" y bidireccional, esta condición garantiza que la suma de los "radios" de las fronteras de exploración es ya mayor o igual al mejor camino encontrado (μ), lo que prueba matemáticamente que es imposible hallar un camino más corto. Ahí, y solo ahí, el algoritmo se detiene, devolviendo el camino óptimo.

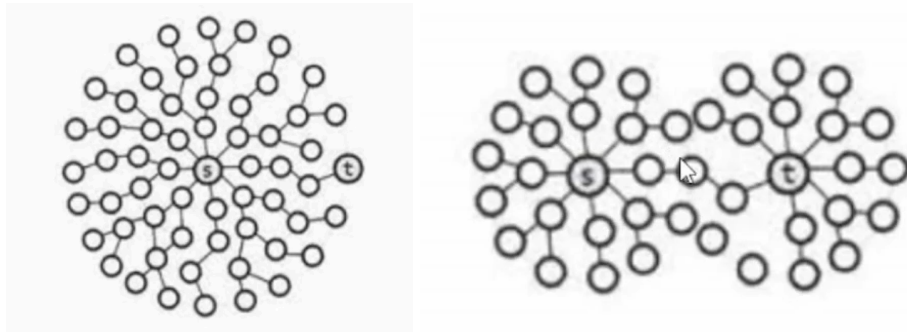
2.5 Complejidad Temporal

La eficiencia del sistema varía drásticamente según el algoritmo seleccionado. La diferencia no es de segundos, sino de escala.

2.5.1 El Problema de la Búsqueda Tradicional (Unidireccional):

Imaginemos que buscamos conectar a un actor novato con Kevin Bacon. Si empezamos solo desde el novato y exploramos todas sus conexiones, luego las conexiones de sus vecinos, y así sucesivamente, el número de personas a revisar aumenta exponencialmente en cada paso. Para solucionar este problema se aplicó una estrategia de Búsqueda Bidireccional tanto para el "Modo Velocidad", que utiliza BFS, como los modos

ponderados, que utilizan Dijkstra. Se inicia la búsqueda desde el novato y desde Kevin Bacon al mismo tiempo. En lugar de intentar cubrir todo el mapa desde un extremo, expandimos dos círculos pequeños hasta que se tocan en el medio. Ninguno de los dos se expande lo que se habría expandido en una búsqueda unidireccional. Representación gráfica:



2.5.2. La Eficiencia de Dijkstra (Modos Ponderados):

Para los modos donde importa la calidad ("Casual" y "Crítico"), no podemos simplemente saltar pasos; tenemos que evaluar el costo de cada película. Para esto, utilizamos una estructura de datos inteligente (Cola de Prioridad) que mantiene siempre las películas más prometedoras al principio de la lista. En lugar de revisar la lista entera cada vez para ver cuál es la mejor opción, el sistema puede "sacar" la mejor opción instantáneamente, permitiendo navegar un mapa complejo de millones de conexiones en menos de un segundo. Convierte una iteración por listas que crece exponencialmente a medida que el grafo crece a un costo apenas logarítmico.

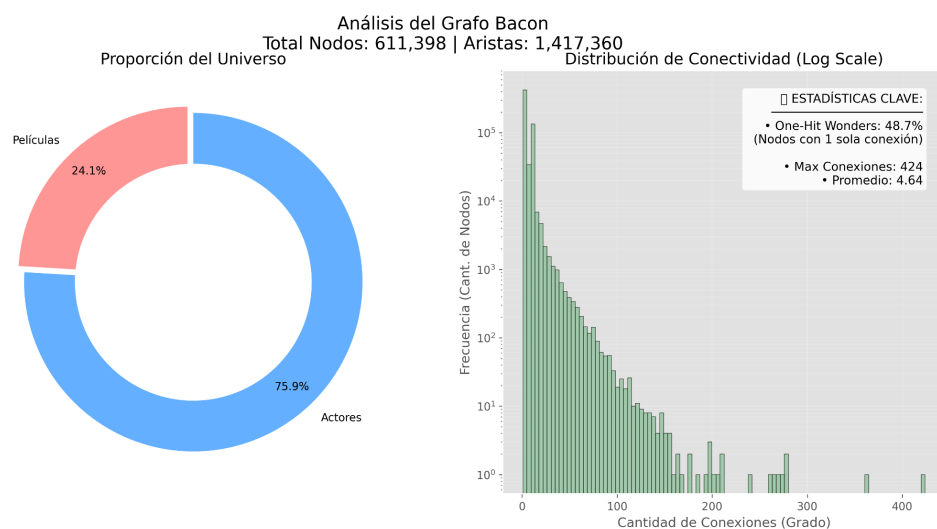
3. Ingeniería de Datos y Estructuras en Memoria (ETL)

El procesamiento de datos constituyó la base del rendimiento del sistema. Se utilizaron cuatro fuentes de datos masivas de IMDb (*title.principals*, *title.basics*, *name.basics*, *title.ratings*) actualizadas hasta el 16/11/2025, fecha donde fueron descargadas. El objetivo fue trabajar con gigabytes de texto que superan la memoria RAM estándar y transformar los datos crudos en información manejable con técnicas de procesamiento por lotes (*chunking*) y estructuras de datos optimizadas.

3.1. Procesamiento por Lotes y Filtrado en Cascada

Debido a que los archivos fuente alcanzan varios gigabytes, se descartó la carga completa en favor de un procesamiento iterativo (*chunking*) utilizando la librería Pandas.

Para ello, se diseñó un pipeline de ETL (*Extract, Transform, Load*) que aplica filtros estrictos en memoria antes de consolidar la estructura del grafo. El proceso comienza restringiendo las categorías exclusivamente a *movie* y *tvMovie*, descartando formatos ajenos al análisis como videojuegos o episodios de TV, para luego aplicar un filtro de calidad basado en la popularidad que admite únicamente nodos con un umbral de votos superior a 100 ($N > 100$), garantizando así una relevancia cultural verificable. Finalmente, se asegura la consistencia topológica del sistema mediante la eliminación de "islas", descartando aquellos actores o películas que, tras los filtros anteriores, carezcan de aristas válidas y queden aislados en la estructura.



3.2. Estructuras de Datos en Memoria (RAM)

Con el objetivo de maximizar la velocidad de los algoritmos de búsqueda y reducir el tiempo de acceso a una complejidad constante de $O(1)$, la arquitectura base del proyecto reside íntegramente en memoria RAM mediante el diseño de cuatro diccionarios (Hash Maps) optimizados. La estructura central, que define la topología de la red, es un diccionario de listas de adyacencia (GRAFO) donde cada clave (ID de IMDb) apunta a una lista de nodos vecinos, permitiendo una navegación inmediata entre entidades. Si la clave es un ID de película, su valor será una lista de ID de actores y viceversa.

Paralelamente, para la gestión de atributos, se implementaron mapas auxiliares de acceso directo. *MAPA_PELICULAS* y *MAPA_ACTORES* funcionan como almacenes de metadatos; el primero utiliza el identificador único *tconst* para recuperar una tupla compacta con valores críticos (título, *rating*, votos) necesarios para el cálculo dinámico de las funciones de costo, mientras que el segundo vincula los identificadores *nconst* con la identidad del actor. Finalmente, para resolver la ambigüedad en la entrada de datos por

parte del usuario, se desarrolló un índice invertido (*MAPA_NOMBRES*) que emplea normalización Unicode (NFKD). Este mapa asocia cadenas de texto estandarizadas (sin acentos ni puntuación) con listas de IDs, permitiendo así el manejo eficiente de homónimos y discrepancias ortográficas sin comprometer la velocidad de búsqueda.

3.3. Persistencia mediante Serialización

Si bien el sistema opera en RAM, la construcción de estas estructuras desde los datos crudos conlleva un costo temporal elevado. Para mitigar esto, se implementó un mecanismo de persistencia binaria utilizando el protocolo *pickle* de Python. El ETL se ejecuta una única vez (o ante actualizaciones de datos), guardando el estado exacto de la memoria en archivos *.pkl*. La aplicación web, al iniciarse, procesa estos archivos directamente a la RAM en tiempo reducido, logrando una disponibilidad casi inmediata.

4. Arquitectura de Software: De Streamlit a Flask

El desarrollo del proyecto atravesó dos etapas marcadas por los requerimientos de personalización y control.

4.1. Primer Intento: Limitaciones de Streamlit

Inicialmente, se desarrolló un prototipo funcional en Streamlit. Si bien esta herramienta permitió validar la lógica de grafos rápidamente usando solo Python, presentó limitaciones críticas para el producto final. Debido a que Streamlit impone su propia estructura visual, implementar el diseño deseado (inspirado en Hollywood y los Oscars) resultaba imposible sin inyecciones de código inestables que intentaban forzar un diseño que el formato por defecto rechaza. Además, funciones complejas como el autocompletado eran irrealizables.

4.2. La Migración a Flask (Arquitectura Web Completa)

Para lograr una experiencia de usuario más personalizada se migró a Flask, un micro-framework que actúa como el nexo entre el procesamiento lógico y la presentación web.

¿Por qué Flask y no Python base? Python estándar no tiene la capacidad nativa de "escuchar" peticiones web (Protocolo HTTP), gestionar puertos o interpretar rutas de navegador. Flask abstrae esta complejidad, permitiendo exponer funciones de Python como "rutas" accesibles vía web. Por otro lado, el servidor Flask está diseñado para ser *stateless*.

Esto significa que no almacena información de usuario entre peticiones. Toda la lógica pesada (el grafo) es consultada al inicio, pero la aplicación no retiene el estado de la sesión, lo que facilita la escalabilidad y el uso eficiente de recursos. Al separar el sistema, el Frontend (Navegador/JavaScript) y el Backend (Servidor/Python) son entornos aislados que no comparten memoria. JSON (JavaScript Object Notation) actúa como el idioma universal de intercambio de mensajes. JavaScript empaqueta la solicitud (IDs de actores, filtros) en un JSON, y Python responde con otro JSON (el camino encontrado).

4.3. Interacción de Tecnologías (El Stack)

La arquitectura final de la aplicación orquesta la coordinación de cuatro lenguajes distintos, estableciendo una clara división entre la presentación visual y la lógica computacional. En el lado del cliente (Frontend), se optó por una estructura semántica en HTML complementada con CSS para el diseño, lo que permitió un control visual de alta precisión ('píxel por píxel') y una estética personalizada imposible de lograr con las plantillas predefinidas de Streamlit.

La dinámica del sistema es gestionada por JavaScript, que actúa como el nexo operativo: detecta los eventos de usuario y administra la comunicación asíncrona con el servidor mediante peticiones *fetch*, evitando la recarga de la página. Una vez que Python —que reside en el servidor (*Backend*)— ejecuta los algoritmos de grafos y resuelve el camino matemático, devuelve la solución en formato JSON. Finalmente, JavaScript intercepta esta respuesta y manipula el DOM para renderizar los resultados en tiempo real, completando así el ciclo de interacción.

5. Consideraciones de Despliegue y Arquitectura Híbrida

El desafío final del proyecto consistió en trasladar la lógica teórica, validada en un entorno de desarrollo local con recursos abundantes (16GB RAM), a un entorno de producción en la nube bajo restricciones estrictas de costos (Render Free Tier, límite físico de 512MB RAM). Esta restricción obligó a un rediseño arquitectónico fundamental, bifurcando el proyecto en dos ramas con estrategias de gestión de memoria opuestas.

5.1. El Dilema: Memoria vs. Persistencia

La versión local ("Main Branch") prioriza la velocidad absoluta cargando la totalidad del grafo y sus metadatos en estructuras hash en memoria RAM. Si bien esto ofrece tiempos de acceso de nanosegundos, el grafo curado y la demás información para el

funcionamiento de la app ocupa más de 600MB al ser instanciado como objetos de Python, provocando el colapso inmediato del contenedor en la nube por desbordamiento de memoria (*OOM Kill*).

5.2. Solución Inicial: Persistencia Relacional (SQL)

Para la versión de producción ("Deploy Branch"), se refactorizó la capa de acceso a datos sustituyendo los diccionarios en memoria por una base de datos **SQLite**, proveyendo a los algoritmos de búsqueda de un cursor de SQL en lugar de un diccionario. Así, el consumo de RAM descendió drásticamente, permitiendo que la aplicación arranque y se mantenga estable dentro del plan gratuito.

5.3. El Cuello de Botella de I/O: Latencia y Timeouts

Si bien la implementación SQL solucionó el problema de memoria, introdujo un problema crítico de latencia (*I/O Bound*). La naturaleza de los algoritmos de grafos implica explorar miles de nodos. En la arquitectura SQL, cada paso de expansión requiere una consulta de lectura en disco *SELECT* para obtener los vecinos. En un entorno de nube con disco compartido de baja velocidad, esta latencia acumulada provocaba que las búsquedas complejas superaran el límite de tiempo de ejecución estándar de los servidores web (30 segundos), resultando en errores de *Worker Timeout*.

Para superar las limitaciones físicas del entorno de producción, se ejecutó una estrategia de optimización híbrida que interviene simultáneamente en el nivel de aplicación y en el de servidor. En primera instancia, tras identificar que la consulta reiterada de metadatos para el cálculo de pesos saturaba el acceso a disco, se implementó una caché de atributos en memoria RAM. Esta técnica precarga los valores numéricos de *rating* y votos, eliminando eficazmente el 50% de las operaciones de entrada/salida (I/O). Complementariamente, se reconfiguró el servidor Gunicorn extendiendo el tiempo de espera (*timeout*) a 120 segundos, lo que permite al sistema gestionar búsquedas profundas de larga duración sin interrumpir el servicio por agotamiento de tiempo.

5.4. Escalabilidad Futura (Propuesta)

Como línea futura de investigación se propone la implementación de Mapeo de Enteros (Integer Mapping). Actualmente, los IDs de IMDb son cadenas de texto (ej. "nm0000102"), que consumen significativa memoria (50-60 bytes por objeto). Sustituir estos identificadores por enteros primitivos de 4 bytes (ej. 102) permitiría comprimir el grafo

completo en arreglos contiguos de memoria, permitiendo regresar a una arquitectura 100% en RAM incluso en servidores de baja capacidad como el de Render y hacer que el proceso sea tan veloz como en local.

6. Validación Experimental y Análisis Estadístico

Para trascender la implementación teórica y verificar la eficacia del sistema en un entorno de grafos masivos, se diseñó y ejecutó un experimento de dos fases: un *Benchmark* de Rendimiento Algorítmico para medir la eficiencia computacional y una Simulación de Montecarlo para analizar el funcionamiento individual de cada modo, además de comparar los resultados con la centralidad teórica del grafo.

6.1. Metodología del Experimento

El experimento se realizó sobre el subgrafo conexo principal de la industria, filtrando actores con un grado de conexión $d \geq 15$ para garantizar búsquedas significativas. Se ejecutaron más de 200.000 simulaciones de búsquedas de caminos entre pares de actores seleccionados aleatoriamente ($u, v \in U$) utilizando la versión del sistema en memoria (RAM) para acelerar el proceso. Finalmente, se trabajó con más de un 1,7 millones de filas, que pertenecen a los actores o películas a los que se visitó en cada búsqueda.

6.2. Resultados de Eficiencia Algorítmica (Benchmark)

Se comparó el rendimiento de los algoritmos de búsqueda Unidireccional vs. Bidireccional bajo condiciones idénticas.

6.2.1. Optimización de BFS

En el "Modo Velocidad" (grafos no ponderados), la diferencia de rendimiento validó empíricamente la reducción del espacio de búsqueda de $O(b^d)$ a $O(B^{d/2})$. Los resultados mostraron una disparidad drástica en la cantidad de nodos visitados: mientras que el enfoque unidireccional requirió explorar un promedio de 136,000 nodos para encontrar una conexión de tres o cuatro grados, la implementación bidireccional convergió visitando apenas 1,400 nodos. Esta reducción del espacio de búsqueda se tradujo en una aceleración promedio de 78 veces, logrando disminuir los tiempos de respuesta de aproximadamente 0.9 segundos a tan solo 0.01 segundos, lo cual confirma la inviabilidad del enfoque tradicional para interacciones en tiempo real.

6.2.2. Optimización de Dijkstra

Para los modos ponderados ("Casual" y "Crítico"), se implementó un Dijkstra Bidireccional con Criterio de Parada Riguroso. A diferencia de implementaciones ingenuas que se detienen al primer encuentro de fronteras (lo que genera resultados subóptimos), nuestro algoritmo continúa la expansión hasta satisfacer la condición $Top(PQ_{Fwd}) + Top(PQ_{Rev}) \geq \mu$, donde μ es el costo del mejor camino hallado hasta el momento. Esta restricción matemática garantiza una optimalidad del 100% respecto al algoritmo de Dijkstra clásico, manteniendo aun así una aceleración significativa de 6.5x en promedio respecto a la búsqueda unidireccional.

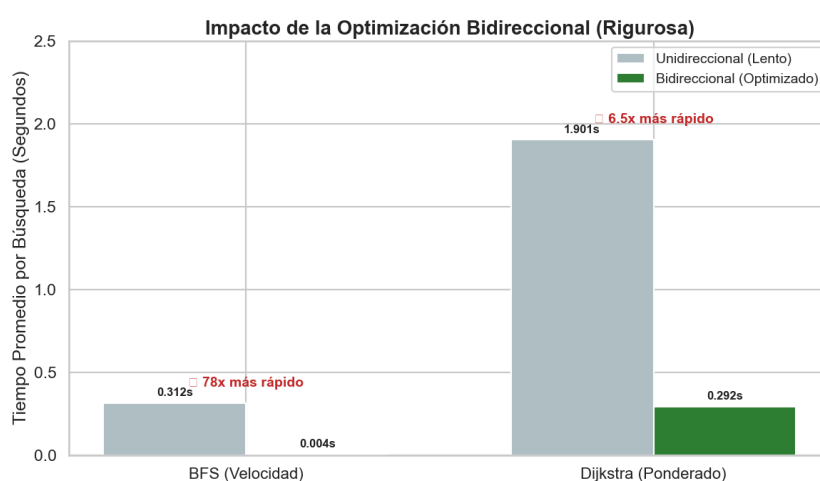


Figura 6.1. Comparativa de rendimiento computacional. Se observa la reducción drástica del tiempo de ejecución gracias a la estrategia bidireccional, logrando una aceleración de 78x en búsquedas no ponderadas y 6.5x en grafos ponderados.

6.3. Análisis de Redes y Tendencias (Resultados de Montecarlo)

Tras procesar más de 100,000 caminos óptimos, se analizaron las características de las películas que actúan como "puentes" en el grafo, revelando cómo las funciones de costo alteran la topología de la red.

6.3.1. Distribución de Calidad (Ratings)

Los datos obtenidos confirman que la función de costo del "Modo Crítico" penaliza exitosamente el contenido de baja calidad. Mientras el "Modo Velocidad" presenta una dispersión de ratings amplia con una mediana cercana a 6,5, utilizando cualquier película disponible para cerrar el camino, el "Modo Crítico" desplaza la mediana significativamente

hacia 7,5 y reduce la varianza. En el “Modo Casual” sucede algo similar, solo que aumenta la varianza. Esto indica que el algoritmo prefiere desviarse por caminos topológicamente más largos si estos transcurren a través de nodos de alta calidad.

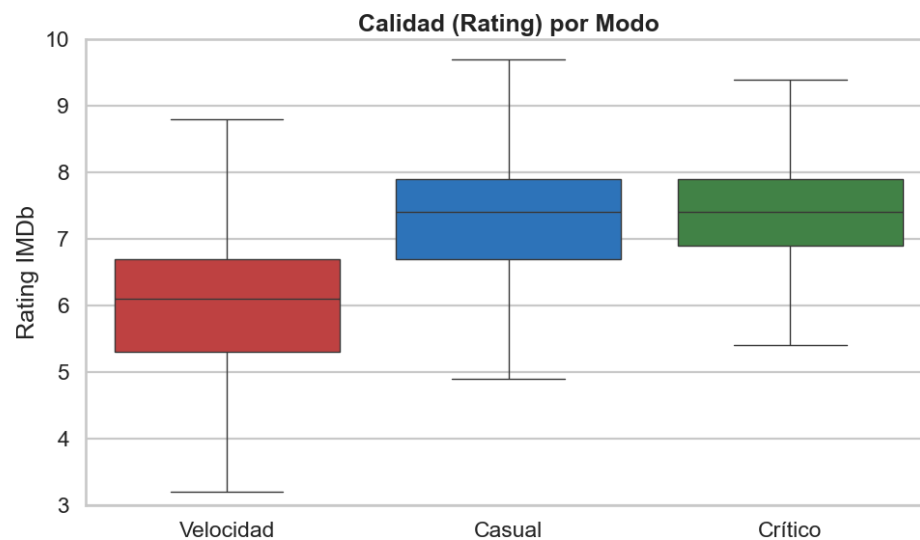


Figura 6.2. Diagrama de caja (*Boxplot*) de los *ratings* promedio por camino. El "Modo Crítico" (verde) desplaza significativamente la mediana hacia calificaciones superiores, filtrando el contenido de baja calidad presente en el "Modo Velocidad".

6.3.2. Sesgo Temporal (Años)

Se observó también un comportamiento emergente respecto a las eras cinematográficas seleccionadas por cada algoritmo. El "Modo Velocidad" muestra una preferencia marcada por el cine contemporáneo (2000-2024), fenómeno que se explica porque los elencos modernos tienden a ser más numerosos (blockbusters), aumentando la probabilidad matemática de conexión. En contraste, el "Modo Crítico" aplanla la curva temporal, recuperando una mayor densidad de películas de las décadas de 1970 y 1990. Esto valida la capacidad del sistema para redescubrir "Clásicos de Culto" que actúan como puentes de calidad entre generaciones, evitando el sesgo de actualidad que si tiene el "Modo Casual".

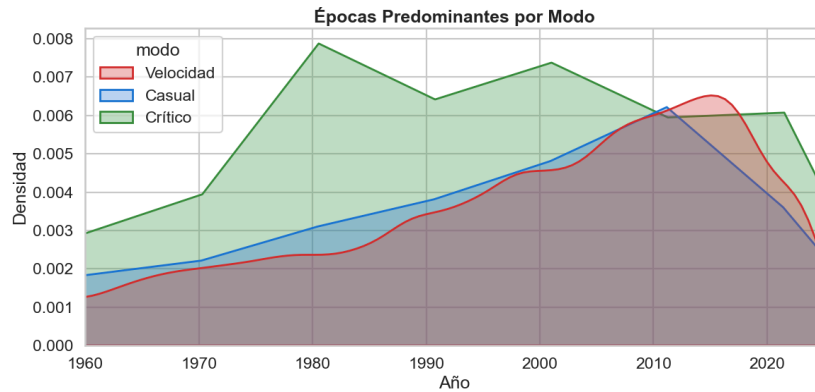


Figura 6.3. Densidad temporal de las películas seleccionadas. Nótese cómo el "Modo Velocidad" (rojo) y "Modo Casual" (azul) presentan un sesgo de actualidad marcado hacia el cine post-2000, mientras que el "Modo Crítico" (verde) distribuye la navegación más equitativamente a lo largo de las décadas.

6.3.3. Centralidad de Intermediación (The "Hubs")

El experimento permitió identificar las películas con mayor Centralidad de Intermediación Empírica (frecuencia de aparición en caminos cortos).

Finalmente, el experimento permitió identificar las películas con mayor Centralidad de Intermediación Empírica. Por un lado, las películas más centrales en la topología del grafo son animadas, ya que registran muchos actores de voces. Sin embargo, en la práctica no se ve lo mismo. Al segmentar por modo aparecen nichos distintivos: en el "Modo Casual", películas como *The Dark Knight*, *The Godfather* o *Inception* ascienden en importancia, mostrando el sesgo que tiene el top 250 de IMDb a películas aclamadas tanto por la audiencia como por la crítica. Por el otro lado, en el "Modo Crítico" aparecen películas documentales (*Henry Fonda: The Man and His Movies*), películas con bajo rating y votos (*As You Like It*) y películas con mucho rating y pocos votos (*My Hero*). Esto estaría demostrando que, quizás, el "Modo Crítico" no funciona tan bien como se esperaba.

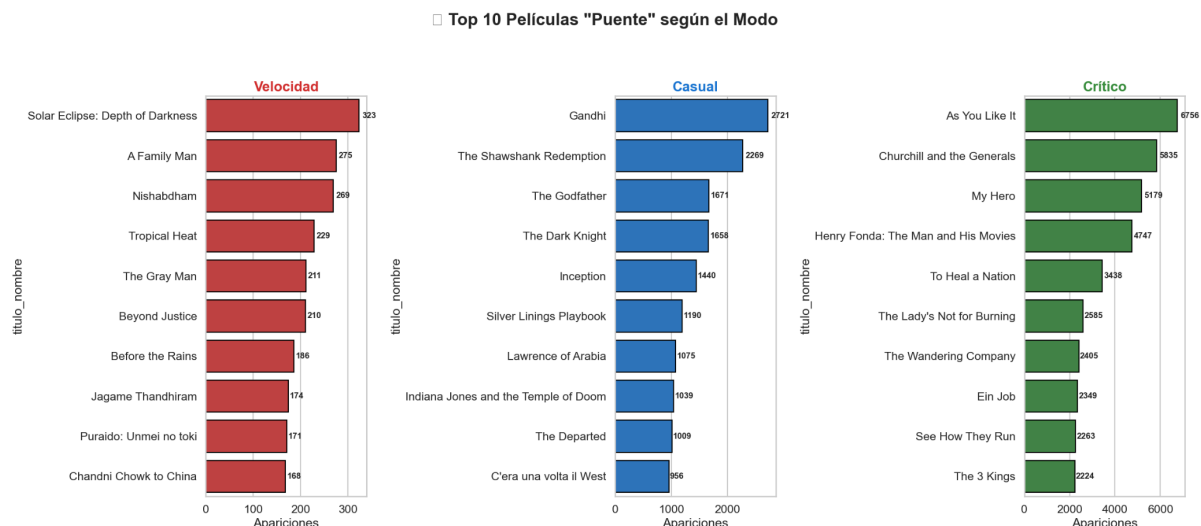


Figura 6.4. Top 10 de películas con mayor centralidad de intermediación (*Betweenness Centrality*) según el perfil de búsqueda. Se evidencia cómo cada algoritmo prioriza conjuntos de películas disjuntos.

6.4. Conclusión del Experimento

La experimentación valida la hipótesis central del trabajo: es posible alterar la percepción de la distancia en una red social mediante la manipulación matemática de los pesos de las aristas. El sistema no solo encuentra conexiones, sino que las cura activamente, ofreciendo rutas que son estadísticamente distintas y semánticamente coherentes con el perfil del usuario (Velocidad vs. Calidad).

7. Conclusión

La matemática es pura abstracción. Es alejarse de lo concreto, de lo tangible, para escribir en un lienzo las reglas de la realidad misma. Este trabajo no busca profundizar en la abstracción sino, todo lo contrario, plasmar la teoría en lo concreto. A través de la lógica computacional y la teoría de grafos, se logró trazar un esquema capaz de explorar millones de conexiones cinematográficas, transformando datos crudos en un mapa cultural navegable.

El desarrollo del proyecto trascendió la mera implementación de algoritmos. Implicó la estructuración de relaciones simétricas y la definición de conjuntos lógicos para dar forma a una idea. Fue un proceso iterativo de ingeniería: desde la limpieza de *Big Data* para garantizar la eficiencia, hasta la superación de las barreras físicas de la computación en la

nube, donde el sistema se vio forzado a escalar, fallar y ser reconstruido bajo nuevas estrategias de optimización de memoria.

Originalmente, el problema se planteaba como la búsqueda de un "hilo invisible" que uniera a toda la industria. No obstante, en una era digital donde el volumen de información (y generación) es infinito y las herramientas están al alcance de todos, la mera existencia de un camino entre dos puntos ha dejado de ser la incógnita principal. Los resultados de la simulación de Montecarlo revelaron que no existe una única industria cinematográfica, sino múltiples topologías superpuestas: nombramos a los *blockbusters* masivos y las obras de culto, pero existen tantos como hay gustos. La verdadera pregunta ya no es *si* existe una conexión, sino *cómo* podemos utilizar nuestro conocimiento para hallar, dentro del ruido de los datos masivos y de todos los caminos posibles, uno que sea relevante.